

Más de tidyverse

Jorge Mario Estrada Alvarez PhD. MSc. FETP

Crear nuevas columnas o modificarlas

En R podemos **crear o modificar variables** a partir de reglas u operaciones, usando:

```
mutate(.data = ,  
       variable_nueva = )# regla de creacion
```

¿Qué hace `mutate()`?

Permite:

- Crear nuevas variables.
- Modificar columnas existentes.
- Realizar operaciones matemáticas o lógicas sobre variables y entre variables
- Puede aplicarse sobre datos nominales (caso a caso).
- También sobre datos agrupados (por ejemplo, por municipio o departamento).

Funciones de apoyo

Función	Uso principal
<code>case_when()</code>	Crear categorías a partir de condiciones
<code>if_else()</code>	Condicional simple (dos opciones)
<code>round()</code> , <code>log()</code> , <code>sqrt()</code>	Operaciones numéricas comunes
<code>as.numeric()</code> , <code>as.character()</code>	Conversión de tipo de variable

y la gran mayoría de funciones que tengan un efecto de cambio o de creación de nueva información a partir del uso de otras variables.

if_else(): condicional simple en R

Se usa para **crear o modificar variables** cuando hay **una sola condición lógica** con dos posibles resultados:

si la condición se cumple → devuelve un valor,

si no → devuelve otro.

```
# Ejemplo práctico: clasificar casos por edad
casos %>%
  mutate(menor_edad = if_else(edad < 18, "Menor", "Adulto"))
```

Características:

- Evalúa una condición lógica (TRUE / FALSE).
- Devuelve un valor para cada caso según el resultado.

Tip

Usa `if_else()` cuando haya una sola condición; si tienes múltiples rangos o categorías, usa `case_when()`.

case_when(): crear columnas con re-categorización

Permite **crear o recodificar variables** a partir de condiciones lógicas múltiples, de forma clara y legible.

SIEMPRE SE USA CON `mutate()`

```
# Estructura

datos %>% # datos
  mutate(nueva_variable = case_when(condicion_1 ~ "respuesta1",
                                     condicion_2 ~ "respuesta2"))
```

```
# Ejemplo práctico: crear grupos de edad
casos %>%
  mutate(grupo_edad = case_when(
    edad < 5 ~ "0-4 años",
    edad < 15 ~ "5-14 años",
    edad < 60 ~ "15-59 años",
```

```
TRUE ~ "60+ años" # Condición por defecto
))
```

Homogeneizar texto con stringr

Las funciones `str_to_lower()`, `str_to_upper()` y `str_to_title()` permiten **estandarizar el formato de texto** en variables de tipo carácter (cadenas).

```
datos %>%
  mutate(
    municipio_min = str_to_lower(Municipio), # todo en minúsculas
    municipio_may = str_to_upper(Municipio), # todo en mayúsculas
    municipio_tit = str_to_title(Municipio)  # primera letra en mayúscula
  )
```

Separar y unir columnas

Las funciones `separate()` y `unite()` permiten **dividir** o **combinar** columnas dentro de un `data.frame`.

```
separate(data = , # dataset
          col = "variable", # variable a separar entre comillas
          into = , # nombre de columnas en la se va a separar
          sep = "") # separador entre comillas

unite(data = , # dataset
       col = , # nueva columna
       c("col1", "col2"), # columnas a unir
       sep = " ") # separador
```

Ejemplo 1: separar una columna

```
datos %>%
  separate(col = "fecha",
           into = c("año", "mes", "día"),
           sep = "-")
```

Ejemplo 2: unir varias columnas

```
datos %>%
  unite(col = "fecha_completa", c("año", "mes", "día"), sep = "-")
```

Eliminar duplicados con `distinct()`

Permite **remover filas duplicadas** en un conjunto de datos, conservando solo los registros **únicos** según las variables indicadas.

```
# Ejemplo 1: eliminar duplicados en todo el dataset
datos %>%
  distinct()

# Ejemplo 2: eliminar duplicados considerando variables específicas
datos %>%
  distinct(nombre, documento, .keep_all = TRUE)
```

Tip

Usa el argumento `.keep_all = TRUE` para conservar todas las columnas del registro original, eliminando solo las repeticiones.